

Guia de estudo e boilerplates

Next.js, TanStack Query, Dexie e shadcn/ui até o fim da Sprint 1

DOJO-L · 10 de setembro de 2026

Guia de estudo e boilerplates

Rastreabilidade: [issue #11 — Dojo D2, Técnico e demonstrações orientadas](#).

Antes de copiar

Este é um **boilerplate didático**, não uma decisão sobre a arquitetura final. Ele conecta uma entidade deliberadamente chamada `RequisitoDojo` e não decide:

- requisito e meta como entidades separadas ou um agregado;
- demanda como entidade ou atributo;
- evidência sem meta vinculada;
- base legal, dados de menores ou direito de imagem.

Use a sequência para aprender as ferramentas. Antes de carregar qualquer parte para produção, confira as decisões técnicas vigentes e o resultado dos spikes.

Os trechos desta página também existem como arquivos compiláveis em `web/src/`, e `web/montar-demo.sh` monta a aplicação que os executa. Eles foram validados com Next.js 16.3.4, React 19.3.0, TanStack Query 5.102.8, Dexie 4.4.6, dexie-react-hooks 4.4.0 e shadcn 4.21.0.

Na aplicação Next.js descartável ou já autorizada:

```
BASH
npm install --save-exact \
  @tanstack/react-query@5.102.8 \
  dexie@4.4.6 \
  dexie-react-hooks@4.4.0
```

Next.js como casca do exemplo

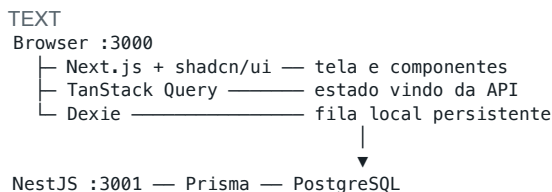
No App Router, páginas e layouts são componentes de servidor por padrão. O `layout` pode renderizar um provider cliente; os hooks, eventos e APIs do navegador ficam em arquivos abaixo de uma fronteira `'use client'`. Assim, a aplicação não precisa virar inteira um componente cliente.

Neste guia:

- `app/layout.tsx` é servidor e compõe o provider;
- `app/providers.tsx` é cliente porque cria estado e contexto;
- a página interativa é cliente porque usa hooks e eventos;
- o componente Dexie é cliente porque IndexedDB só existe no navegador.

Fonte: [Next.js — Server and Client Components](#), acesso em 10/09/2026.

Mapa do exemplo



L-01 · Provider do TanStack Query

`app/providers.tsx`:

TSX

```
'use client';

import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { useState, type ReactNode } from 'react';

export function Providers({ children }: { children: ReactNode }) {
  const [queryClient] = useState(() => new QueryClient());
  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  );
}
```

app/layout.tsx:

```
TSX
import { Providers } from './providers';

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="pt-BR">
      <body><Providers>{children}</Providers></body>
    </html>
  );
}
```

Por que o cliente nasce dentro de useState: ele não é recriado a cada renderização do provider.

L-02 · Cliente HTTP que não esconde 400 e 500

lib/api.ts:

```
TS
const API_URL = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3001';

export async function api<T>(path: string, init?: RequestInit): Promise<T> {
  const response = await fetch(`${API_URL}${path}`, init);

  if (!response.ok) {
    const detail = await response.text();
    throw new Error(`HTTP ${response.status}: ${detail}`);
  }

  return response.json() as Promise<T>;
}
```

.env.local da aplicação Next.js:

```
DOTENV
NEXT_PUBLIC_API_URL=http://localhost:3001
```

L-03 · Hooks de leitura e criação

hooks/use-requisitos-dojo.ts:

```
TSX
'use client';

import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query';
import { api } from '@lib/api';

export interface RequisitoDojo {
  id: string;
  origem: string;
  indicador: string;
  parametroAfericao: string;
  formaVerificacao: string;
  frequenciaApuracao: string;
}

export type CreateRequisitoDojoDto = Omit<RequisitoDojo, 'id'>;

export function useRequisitosDojo() {
  return useQuery({
    queryKey: ['requisitos-dojo'],
  });
}
```

```

    queryFn: () => api<RequisitoDojo[]>('/requisitos-dojos'),
  });
}

export function useCriarRequisitoDojo() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (dto: CreateRequisitoDojoDto) =>
      api<RequisitoDojo>('/requisitos-dojos', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(dto),
      }),
    onSuccess: () =>
      queryClient.invalidateQueries({ queryKey: ['requisitos-dojos'] }),
  });
}

```

L-04 · Tela cliente com estados explícitos

```

TSX
'use client';

import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { OutboxDojo } from '@components/outbox-dojos';
import { useCriarRequisitoDojo, useRequisitosDojo } from '@hooks/use-requisitos-dojos';

export default function RequisitosDojoPage() {
  const requisitos = useRequisitosDojo();
  const criar = useCriarRequisitoDojo();

  if (requisitos.isPending) return <p>Carregando...</p>;
  if (requisitos.isError) return <p>Falha: {requisitos.error.message}</p>;

  return (
    <main className="mx-auto max-w-xl space-y-4 p-6">
      <form
        className="flex gap-2"
        onSubmit={(event) => {
          event.preventDefault();
          const form = new FormData(event.currentTarget);
          criar.mutate({
            origem: String(form.get('origem')),
            indicador: 'indicador didático',
            parametroAfericao: 'parâmetro didático',
            formaVerificacao: 'verificação didática',
            frequenciaApuracao: 'frequência didática',
          });
        }}
      >
        <Input name="origem" aria-label="Origem" />
        <Button type="submit" disabled={criar.isPending}>Criar</Button>
      </form>

      {criar.isError && <p>Falha ao criar: {criar.error.message}</p>}

      <ul>
        {requisitos.data.map((item) => <li key={item.id}>{item.origem}</li>)}
      </ul>

      <OutboxDojo />
    </main>
  );
}

```

Os textos “didático” evitam que o exemplo pareça confirmação de regra do Instituto.

L-05 · Banco local e fila Dexie

lib/db.ts:

```

TS
import Dexie, { type EntityTable } from 'dexie';

export interface ItemOutboxDojo {
  id?: number;
  clientMutationId: string;
}

```

```

    payload: string;
    estado: 'pendente' | 'enviado';
    criadoEm: number;
  }

export const db = new Dexie('cybersetor_dojo') as Dexie & {
  outbox: EntityTable<ItemOutboxDojo, 'id'>;
};

db.version(1).stores({
  outbox: '++id,&clientMutationId,estado,criadoEm',
});

```

components/outbox-dojo.tsx:

```

TSX
'use client';

import { useLiveQuery } from 'dexie-react-hooks';
import { db } from '@lib/db';

export function OutboxDojo() {
  const pendentes = useLiveQuery(
    () => db.outbox.where('estado').equals('pendente').toArray(),
    [],
    [],
  );

  async function adicionar() {
    await db.outbox.add({
      clientMutationId: crypto.randomUUID(),
      payload: JSON.stringify({ exemplo: true }),
      estado: 'pendente',
      criadoEm: Date.now(),
    });
  }

  async function marcarPrimeiroComoEnviado() {
    const primeiro = pendentes[0];
    if (primeiro?.id !== undefined) {
      await db.outbox.update(primeiro.id, { estado: 'enviado' });
    }
  }

  return (
    <section>
      <p>Pendentes: {pendentes.length}</p>
      <button onClick={adicionar}>Adicionar</button>
      <button onClick={marcarPrimeiroComoEnviado}>Marcar enviado</button>
    </section>
  );
}

```

L-06 · O que precisa existir no back-end

Para o exemplo de TanStack funcionar, a API precisa expor leitura e criação:

```

TS
@Get()
listar() {
  return this.requisitos.listar();
}

@Post()
criar(@Body() dto: CreateRequisitoDojoDto) {
  return this.requisitos.criar(dto);
}

```

No ambiente didático, Next.js usa 3000 e NestJS usa 3001. O bootstrap do Nest habilita CORS apenas para <http://localhost:3000>. Isso é configuração local do exercício, não política de produção.

L-07 · shadcn/ui no repositório existente

Execute somente dentro da aplicação Next.js já autorizada:

BASH

```
npx shadcn@4.21.0 init --defaults --no-monorepo
npx shadcn@4.21.0 add button input --yes
```

Verifique:

- `components/ui/button.tsx` e `components/ui/input.tsx` existem;
- imports usados pela CLI resolvem;
- o formulário de L-04 renderiza;
- alterar o componente muda o código da própria aplicação.

Não abra outro repositório e não execute inicialização no código publicado sem autorização humana.

L-08 · Percurso individual até 22/09

Janela	Foco	Evidência objetiva
11-13/09	repetir NestJS/Prisma e Dexie	cada pessoa explica 201/400/GET e a contagem local 2→1
14-16/09	ligar provider, hooks e shadcn/ui	lista atualiza sem F5 e erro HTTP aparece como erro
17-18/09	redesenhar o mapa integrado	cada pessoa explica as seis fronteiras sem consultar
21/09	recuperação final	bloqueios restantes têm contexto, tentativa e próximo passo

Essas janelas são checkpoints de aprendizagem, não pontos nem estimativas de história.

Até a Review de 22/09, cada evidência é produzida individualmente. O aprofundamento em pares não é outro dojo nem uma recuperação deste material: começa no desenvolvimento do produto durante a Sprint 2, depois da ratificação da mudança de planejamento.

L-09 · Como pedir ajuda no canal da equipe

Envie quatro elementos:

1. **contexto:** arquivo e objetivo;
2. **tentativa:** comando ou trecho mínimo;
3. **resultado real:** erro completo e momento em que ocorreu;
4. **resultado esperado:** o que deveria ter acontecido.

Exemplo:

Contexto: useCriarRequisitoDojo, tentando criar e atualizar a lista. A chamada devolve 400 e o onSuccess não roda. DTO enviado e mensagem completa abaixo. Eu esperava ver o erro no formulário. O que devo observar primeiro?

Evite “não funciona” e prints sem texto copiável.

L-10 · Critério de conclusão individual

A pessoa conclui o percurso quando consegue:

- localizar provider, query, mutation e tratamento de erro;
- explicar por que a consulta usa GET e a criação usa POST;
- provar que a fila Dexie sobrevive ao reload e não indexa booleano;
- identificar os componentes shadcn como arquivos-fonte;
- dizer qual estado pertence ao browser e qual pertence ao servidor;
- nomear uma decisão em aberto que o boilerplate não resolve.

Não há nota composta. Cada evidência é passa, falha ou não executado, com oportunidade de repetir e pedir apoio no canal.

Fontes primárias de ferramenta

- [Dexie — Indexable Type](#)
- [Dexie — Version.stores\(\)](#)
- [Dexie React Hooks — useLiveQuery\(\)](#)
- [TanStack Query — Quick Start](#)
- [TanStack Query — SSR e Next.js](#)
- [Next.js — Server and Client Components](#)
- [MDN — fetch\(\)](#)
- [shadcn/ui — CLI](#)

Consulta registrada em 10/09/2026.